```
class X {
public:
    virtual int g(double x) {
        h(); cout << "X::g" << endl;
        return 1;
    }
    void h() { t(); cout << "X::h" << endl; }
    virtual void t() { cout << "X::t" << endl; }
};
class Z : public X {
public:
    int g(double y) {
        h(); cout << "Z::g" << endl;
        return 3;
    }
    virtual void h() { t(1);  cout << "Z::h" << endl; }
    virtual void t(int k) { cout << "Z::t" << endl; }
};
void f(X& b) { b.g(1); }
int main() {
    X a; Z b; X* p = &b;
    p->g(1.5); //Z::h Z::t Z::h
    p->h(); //X::h X::t
    p->t(); //X::t
    cout << endl;
    f(b); //Z::h Z::t Z::h
}
```

Шаблон класса

```
template <typename T>
class vector {
    T* arr;
    size_t sz;
    size_t cap;
};
```

При объявлении генерируются разные классы для разных типов

```
int main () {
    vector<int> A;
    vector<char> B;
    vector<double> C = A;  //Будет ошибка
}
```

Нетиповой параметр шаблона (у функций также может использоваться)

Нетиповой параметр шаблона должен быть целочисленным значением, известным на стадии компиляции. Начиная с C++20 не только целочисленным.

```
template <typename T, size_t N>
class array {
    T arr[N];
};


int main () {
    array<int, 20> A;
    array<char, 100> B;
}
```

Аргументы шаблона по умолчанию (у функций также разрешены)

```
template<typename T = int, size_t N = 100> // Значения по умолчанию
struct Stack {
    T data[N]; // фиксированная емкость N
    size_t count;
        void push(const T& d){
    }
    // и т. д.
};


int main() {
    Stack i;
    std::cout << typeid(i.data[0]).name() << '\n'; //i - integer
    Stack<double> d;
    std::cout << typeid(d.data[0]).name() << '\n'; //d - double
}
```

Параметризация типов на примере

```
/классическое определение шаблонного класса
template <size_t N, size_t M, typename T = double>
class Matrix {
    T matrix[N][M];
public:
    void print(T inf){
        matrix[0][0] = inf;
        std::cout << matrix[0][0] << '\n';
    }
};
//шаблонный using
template <size_t N, typename T = double>
using SMatrix = Matrix<N,N,T>;


//шаблонная функция, использующая шаблонный класс
template <size_t N, size_t M, size_t K, typename T = double>
Matrix<M,N,T> operator* (const Matrix<M,K,T>& left, const Matrix<K,N,T>& right){
    //пока не инстанцировали, тут даже не будет проверки на корректность возвращаемых переменных
    //пустой код тоже скомпилируется
}


int main () {
    Matrix<5,6> rectangular;
    SMatrix<5,int> quadratic;
    rectangular.print(4.9);
    quadratic.print(3);
}
```

Специализация шаблона класса

```
template <typename T, typename U>
class C {
public:
    void f (){ std::cout << "Primary\n";}
};

template<typename U>
class C <int, U> {
public:
    void f(){ std::cout << "T = int\n"; }
};

template<typename T>
class C<T, double> {
public:
    void f() { std::cout << "U = double\n"; }
};
```

```
template<typename T, typename U>
class C<T*, U> {
public:
    void f() { std::cout << "T* used\n"; }
};

template<typename T, typename U>
class C<T, U*> {
public:
    void f() { std::cout << "U* used\n"; }
};

template<typename T, typename U>
class C<T*, U*> {
public:
    void f() { std::cout << "T* and U*\n";}
};
```

```
int main () {
    C<float,int>().f();
    C<int,float>().f();
    C<float,double>().f();
```

```
    C<float,float>().f();
    C<float*,float>().f();
    C<float,float*>().f();
}
```

Частичная специализация методов классов запрещена

```
template <typename T, typename U>
struct C {
    void f(T a, U b);
};

template <typename T, typename U>
void C<T, U>:: f(T a, U b){
    a = b;
}

template <typename T>
void C<T, int>:: f(T a, int b){
    a = b;
}

template <>
void C<int, int>:: f(int a, int b){
    a = b;
}
```

Nested name specifier 'C<T, int>::' for declaration does not refer into a class, class template or class template partial specialization

2 Use of undeclared identifier 'a'

Шаблоны как параметры шаблонов

```
template<typename T>
class Array {
    T *data;
    size_t capacity;
    size_t count;
public:
    Array(): count(0), data(new T[capacity = 10]) { }
    ~Array() { delete [] data; }
    T* begin() { return data; }
    T* end() { return data + count;}
};

template<typename T, template<typename> typename Seq>
class Container {
    Seq<T> seq;
public:
    T* begin() { return seq.begin();}
    T* end() { return seq.end(); }
};

int main () {
    Container<int, Array> container;
    int* p = container.begin();
    while(p != container.end()) {
        std::cout << *p++ ;
    }
}
```

То есть Seq – это шаблонный класс, который принимает один шаблонный параметр

Соответственно, этот тип Seq может быть инстанцирован каким-то шаблонным классом, который принимает один параметр

Шаблоны как параметры шаблонов

До C++17 typename использовался только для параметров-типов, а для шаблонных параметров было зарезервировано слово class:

```
template<typename T, template<typename> class Seq>
```

Сейчас уже можно и так, и так:

```
template<typename T, template<typename> class Seq> // старый стиль
template<typename T, template<typename> typename Seq> // с C++17
```

Если передавать Array<int>- передаётся уже конкретный инстанцированный тип. Тогда синтаксис будет другим (принимаем обычный параметр шаблона):

```
template<typename T>
class Array {
    T *data;
    size_t capacity;
    size_t count;
public:
    Array(): count(0), data(new T[capacity = 10]) { }
    ~Array() { delete [] data; }
    T* begin() { return data; }
    T* end() { return data + count;}
};

template<typename T, typename Seq>
class Container {
    Seq seq;
public:
    T* begin() { return seq.begin();}
    T* end() { return seq.end(); }
};

int main () {
    Container<int, Array<int>> container;
    int* p = container.begin();
    while(p != container.end()) {
        std::cout << *p++ ;
    }
}
```

Вложенные шаблоны

```
template<typename T>
class Complex {
    T var;
public:
    Complex(T var):var(var){}
    template <typename X> Complex(const Complex<X>& c);
};


template<typename T>
template<typename X>
Complex<T>:: Complex(const Complex<X>& c) {/*Тело определения... */}


int main () {
    Complex<int> c(6);
    Complex<double> c1(c);
}
```

Друзья шаблонных классов. Постановка задачи. Тут будет ошибка:

```
template<class INF>
class MyStack {
    using Node = ListNode<INF, MyStack<INF>>;
    Node* top;
    //...

friend std::ostream& operator<<(std::ostream& os, const MyStack<INF>& st);
};

template<class INF>
std::ostream& operator<<(std::ostream& os, const MyStack<INF>& st) {
 . . .
}
```

Потому что здесь внутри класса объявлена обычная (нешаблонная) функция-друг, а снаружи определён шаблон функции. Компилятор видит два разных объекта и не может их связать. Компилятор ожидает найти определение этой конкретной функции, а не шаблон, который мог бы её создать.

Друзья шаблонных классов.
Вариант 1. Определение внутри класса

```
template<class INF>
class MyStack {
    using Node = ListNode<INF, MyStack<INF>>;
    Node* top;
    //...

friend std::ostream& operator<<(std::ostream& os, const MyStack<INF>& st) {
    Node* p = st.top;
    //...
    return os;
}
};
```

Здесь оператор является простой функцией. При каждой специализации класса создаётся новая версия оператора, которая получает аргумент с типом текущей специализации.

Друзья шаблонных классов.
Вариант 2. Определение вне класса

Когда дружественная функция реализована вне класса, она в качестве параметра должна принимать наш шаблонный класс. Таким образом, она будет специализирована для нашего класса. Так как это специализация, следует выполнить предопределение функции.

```
template<typename T> class MyStack; // pre-declare the template class itself
template<typename T> std::ostream& operator<< (std::ostream& o, const MyStack<T>& x);

template<class INF>
class MyStack {
    using Node = ListNode<INF, MyStack<INF>>;
    Node* top;
    //...
    friend std::ostream& operator<< <INF>(std::ostream& os, const MyStack<INF>& st);
};

template<class INF>
std::ostream& operator<<(std::ostream& os, const MyStack<INF>& st) {
    ListNode<INF, MyStack<INF>>* p = st.top;
    //...
    return os;
}
```

Подробнее про вариант 2

Второй вариант считается более безопасным. Такой синтаксис

```
friend std::ostream& operator<< <INF>(std::ostream& os, const MyStack<INF>& st);
```

означает, что другом является только оператор вывода для того же типа INF. То есть мы прописываем, что уже специализация оператора для нашего шаблона INF будет являться нам другом.

Зачем предопределение?

Компилятор видит operator<< с угловыми скобками, значит, нужен вызов шаблонной функции, но нигде нет объявления operator<< как шаблонной функции. Поэтому необходимо предопределить этот уже шаблонный оператор, который будет потом в дальнейшем реализован для нашего класса.

```
template<typename T> class MyStack; // pre-declare the template class itself
template<typename T> std::ostream& operator<< (std::ostream& o, const MyStack<T>& x);
```

Друзья шаблонных классов.
Вариант 3. Все экземпляры шаблона объявляются дружественными

```
template<class INF>
class MyStack {
    using Node = ListNode<INF, MyStack<INF>>;
    Node* top;
public:
    //...
    template <typename U>
    friend std::ostream& operator<< (std::ostream& os, const MyStack<U>& st);
};

template<class INF>
std::ostream& operator<<(std::ostream& os, const MyStack<INF>& st) {
    //...
}
```

То есть каждая специализация шаблона-друга является другом каждой специализации шаблона-класса. Иными словами, любая специализация operator << будет другом нашему классу.

Подробнее про вариант 3 (сгенерированный ИИ наглядный пример)

```
template<typename T>
class BankAccount {
    double balance = 1000000;

    template<typename U>
    friend void hack(const BankAccount<U>& account);
};

template<typename T>
void hack(const BankAccount<T>& account) {
    // Может прочитать balance ЛЮБОГО BankAccount
    std::cout << account.balance << std::endl;
}

int main() {
    BankAccount<int> myAccount;
    BankAccount<double> hackerAccount;
    // hackerAccount (тип double) имеет доступ
    // к приватным полям myAccount (тип int)
    hack(myAccount);
}
```

Компиляция проекта при наличии шаблонов

Принцип раздельной компиляции у проектов, использующих шаблоны, неприменим. То есть нельзя разделить объявление и определение шаблонных классов (функций) в соответствующие файлы hpp и cpp.

Почему?

Шаблоны не являются полноценным кодом, только на этапе компиляции формируется код на основании определения параметров шаблона (инстанцирование).

Но каждый cpp файл после препроцессинга компилируется независимо от других как отдельная единица трансляции в объектный файл. Таким образом, после получения объектного файла реального определения функции в нём нет, потому как непонятно, чем её инстанцировать.

Компиляция шаблона

На этапе линковки происходит поиск соответствующей функции, но нигде подходящей функции нет, будет ошибка.

```
#include "Test.hpp"
template <typename T>
T f (T a, T& b){
    b+=a;
    return b;
}
```

```
template <typename T>
T f (T a, T& b);
template <typename T>
T f (T a, T& b){
    b+=a;
    return b;}
```

Test.o

```
template <typename T>
T f (T a, T& b);
```

Test.hpp

```
#include "Test.hpp"
int main () {
    int a = 6;
    int b = 3;
    f<int>(a,b);
}
```

```
int f (int a, int & b);
int main () {
    int a = 6;
    int b = 3;
    f<int>(a,b);
}
```

main.o

```
Undefined symbols
  "int f<int>(int, int&)", referenced
from: _main in main.o
```

Решение #1

В файле cpp прописывать все возможные реализации, произвести, так называемую, ручную генерацию специализаций (это самый медленный и неоптимальный путь)

```
#include "Test.hpp»

template int f<int>(int,int&);
template double f<double>(double,double&);

template <typename T>

T f (T a, T& b){
    b+=a;
    return b;
}
```

Решение #2

Использовать tpp. И в h-файле написать в конце #include “Test.tpp”

```
template <typename T>
T f (T a, T& b);

#include “Test.tpp"
```

Как вы думаете, что будет, если вместо tpp использовать zzz? ☺

По-настоящему, разницы нет, какой файл включать. Препроцессор раскрывает include, и компилятор получает h-файл с объявлением и определением (см. Решение #3)

Решение #3

Объявление и определение прописывать в одном заголовочном файле hpp (см. cppreference.com)

Test.hpp:

```
template <typename T>
T f (T a, T& b);

template <typename T>
T f (T a, T& b){
b+=a;
return b;
}
```

При этом пустой cpp-файл, имеющий единственную строку *“#include Test.hpp”*, очевидно, не нужен, но если очень хочется, см. дальше

Особенности специализации

Специализированный метод класса или специализированная шаблонная функция – это полноценная функция с вполне себе полноценным определением, она УЖЕ инстанцирована. Если специализация выполнена в h-файле, но при этом есть ещё и cpp, пусть даже пустой, то при компиляции получается на выходе единица трансляции с определением функции.

Таким образом, при очередном включении этого h-файла в другую единицу трансляции, там также сформируется определение этой же функции. То есть у одной функции получится множество определений, что противоречит принципу One Definition Rule, ODR.

Решение #1

Не использовать cpp, а в hpp прописать защиту от повторных включений.

Решение #2

Специализацию функций выносить в cpp. В заголовочном файле останется объявление (объявлений, как известно, может быть очень много), а реализация будет одна.

Решение #3

Использовать inline, таким образом вписывая реализацию функции непосредственно в код, что защищает от ошибки повторного определения.

Использование шаблонов

Пример шаблонного класса

```
template <typename T, size_t N = 5>
class MyVector {
	T * array;
	size_t capacity = N;
	size_t size = 0;
public:
	MyVector(): array(new T [N]) {}
	MyVector(T el): array(new T [N]) {
		AddElement(el);
	}
	void AddElement (T el) {
		//if (size == capacity) resize()
		array[size++] = el;
	}
	T& operator [] (int i) const {
		return array[i];
	}
	virtual ~MyVector() {
		delete [] array;
	}
};
```

Наследование шаблонного класса происходит также, как и обычного класса

```
template <typename T, size_t N = 20>
class MyVector {
        // . . .
};

template <typename T>
class MySet: public MyVector<T> {
public:
        using MyVector<T>::MyVector;
        void AddElement (T el) {
                MyVector<T>::AddElement(el);
        }
};
int main () {
        MyVector<int> vec;
        vec.AddElement(4);
        std::cout << vec[0] << std::endl;

        MySet<double> set(13);
        set.AddElement(5);
        std::cout << set[0] << ' ' << set[1] <<std::endl;
}
```

```
template <typename T, typename U, size_t N = 20>
class MyVector {
public:
    void f () {std::cout << N; }
};
template <size_t N, typename U>
class MyVector<U, char, N> {
public:
    void f () {std::cout << N + 2; }
};
template <size_t N, typename U>
class MyVector<char, U, N> {
public:
    void f () {std::cout << N + 3; }
};

int main () {
    MyVector<double, int> f;
    f.f();
    MyVector<char, int> f1;
    f1.f();
    MyVector<int, char> f2;
    f2.f();
}
```

Частичная специализация

Частичная специализация класса

```
template <size_t N> //нельзя уже задать по умолчанию
class MyVector<char*, N> {
        char** array;
        size_t size = 0;
        int maxLen = N;
public:
        MyVector(): array(new char* [N]) {}
        void AddElement (char* el) {
                //if (size == maxLen) resize()
                array[size] = new char [strlen(el)+1];
                strcpy(array[size], el);
                size++;
        }
        friend std::ostream& operator << (std::ostream& out, const MyVector& vector) {
                for (size_t i = 0; i < vector.size; i++)
                        out << vector.array[i] << ' ‘;
                return out;
        }
        virtual ~MyVector() {
                for (size_t i = 0; i < size; i++)
                        delete [] array[i];
                delete [] array;
        }
};
```

```
int main () {
        char* string = new char[]{"Test"};
        MySet<char*> set1;
        set1.AddElement(string);

        delete [] string;
        std::cout << set1;
}
```

Полная специализация на примере класса наследника

```
template <typename T>
class MySet: public MyVector<T> {
public:
    using MyVector<T>::MyVector;
    void AddElement (T el) {
        MyVector<T>::AddElement(el);
    }
};
```

```
template <>
class MySet<char*> {
public:
    void AddElement (char * el) {
        std::cout << "这是一个空的函数" << std::endl;
    }
};
```

Здесь специализация у класса MySet никак не связана с классом MyVector

```
template <>
class MySet<char*>: public MyVector<char*> {
public:
    void AddElement (char * el) {
        std::cout << "这还是空的函数" << std::endl;
    }
};
```

Здесь специализация MySet уже наследуется от MyVector. **!!! Двух одинаковых специализаций быть, очевидно, не может.**

Специализация метода класса (частичная запрещена)

```
template <typename T, size_t N = 20>
class MyVector {
    T* array;
    size_t capacity = N;
    size_t size = 0;
public:
    //. . .
    void AddElement (T el) {
        //if (size == capacity) resize()
        array[size++] = el;
    }
    //. . .
};

template<>
void MyVector<char*>::AddElement(char* el) {
    //if (size == capacity) resize()
    array[size] = new char [strlen(el) + 1];
    strcpy(array[size], el);
    size++;
}
```

Параметр шаблона может быть пользовательским типом

```
template <typename T, size_t N = 20>
class MyVector {
    //. . .
};
template <typename T>
class MySet: public MyVector<T> {
    //. . .
};
struct Class {
    char name[25];
    static int score;
};

int Class::score = 0;

int main () {
    MyVector<MySet<int>> ms;
    MySet<Class> st;
}
```

Шаблоны как параметры шаблонов

```
#include <vector>
template <typename T>
class MyStack { };

template <typename T, size_t N = 20>
class MyVector { };

template <typename T>
class MySet: public MyVector<T> { };

template <typename T, template <typename> typename Sequence>
class Container {
    Sequence<T> container;
    // что-то . . .
};
int main () {
        Container<double, MyStack> containStack;
        Container<int, MySet> containSet;
        Container<char, MyVector> containVector;    //MyVector имеет два шаблонных параметра
        Container<char, std::vector> containVectorStd; //И std::vector тоже
}
```

error: template template argument has different template parameters than its corresponding template template parameter

Можно сделать так…

```
template <typename T, size_t N = 4>
class MyStack { };

template <typename T, size_t N = 20>
class MyVector {
public:
    void f () {std::cout << N; }
};

template <typename T, size_t N = 4>
class MySet: public MyVector<T> { };

template <typename T, template <typename, size_t> typename Sequence, size_t N = 2>
class MyContainer {
public:
    Sequence<T,N> container;
    // что-то . . .
};

int main () {
    MyContainer<double, MyStack> containStack;
    MyContainer<int, MySet> containSet;
    MyContainer<char, MyVector> containVector;
    containVector.container.f(); //!!!
    // Container<char, std::vector> containVectorStd; //Всё ещё нельзя
}
```

А можно так...

```
#include <vector>

template <typename T, size_t N = 20>
class MyVector { };

template <typename T>
using MyVectorDefault = MyVector <T>;

template <typename T>
using MyVectorStd = std::vector<T>;

template <typename T, template <typename> typename Sequence>
class Container {
    Sequence<T> container;
    // что-то . . .
};

int main () {
    Container<char, MyVectorDefault> containVector;
    Container<char, MyVectorStd> containVectorStd;
}
```

Можно запретить специализацию

```
template<typename T>
void f1(T) {std::cout << 1;} //общий шаблон

template<>
void f1(int) {std::cout << 2;} //специализация для int

template<>
void f1(char) = delete; //специализация для char (запрещено)

int main (){
    char s = 10;
    f1(s);
}
```

candidate function [with T = char] has been implicitly deleted

Как и разрешить только специализацию

```
template<typename T>
void f1(T) = delete; //общий шаблон (запрещаем)

template<>
void f1(int) {std:: cout << 2;} //специализация для int

template<>
void f1(short) {std::cout << 3;} //специализация для short

int main (){
    char s = 10;
    f1(s);
}
```

candidate function [with T = char] has been explicitly deleted

А если шаблоны классов?

Так просто сделать уже не получится, но тоже можно. Один из вариантов прописать специализацию без определения.

```
template <typename T>
class MySet: public MyVector<T> {
public:
    using MyVector<T>::MyVector;
    void AddElement (T el) {
    MyVector<T>::AddElement(el);
        std::cout << "Hello";
    }
    void print() {std::cout << "Set";}
};
template <>
class MySet<char*>;
int main (){
    MySet<int> i;
    MySet<char*> t;//variable has incomplete type 'MySet<char *>'
}
```

Аналогично можно запретить «общий» шаблон.